



REACH

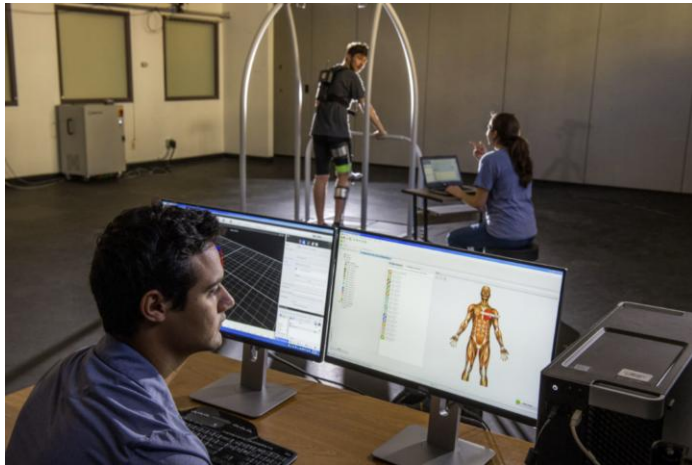
Reinforcement-learning
for Enhanced Arm Control
in Humans

Members: TJ Davis, Lucas Larson, Victor Rodriguez, Clayton Ramsey

Clients: Dr. Zach Lerner & Dr. Carlo R. da Cunha

Mentor: Bailey Hall

Why REACH?



Domain: Simulation and RL for robotics

Stakeholders: NAU Biomechatronics Lab -
reproducible training & evaluation across tasks

Who: Biomechatronics Lab - Dr. Zach Lerner, Associate Professor, Mechanical Engineering, NAU & Dr Carlo Da Cunha, Assistant Professor, Electrical Engineering, NAU

Problem: Robotic development relies on safe and natural interactions; Iterations are long and prototyping is a big commitment; Traditional control methods lack adaptability and perception of human intent

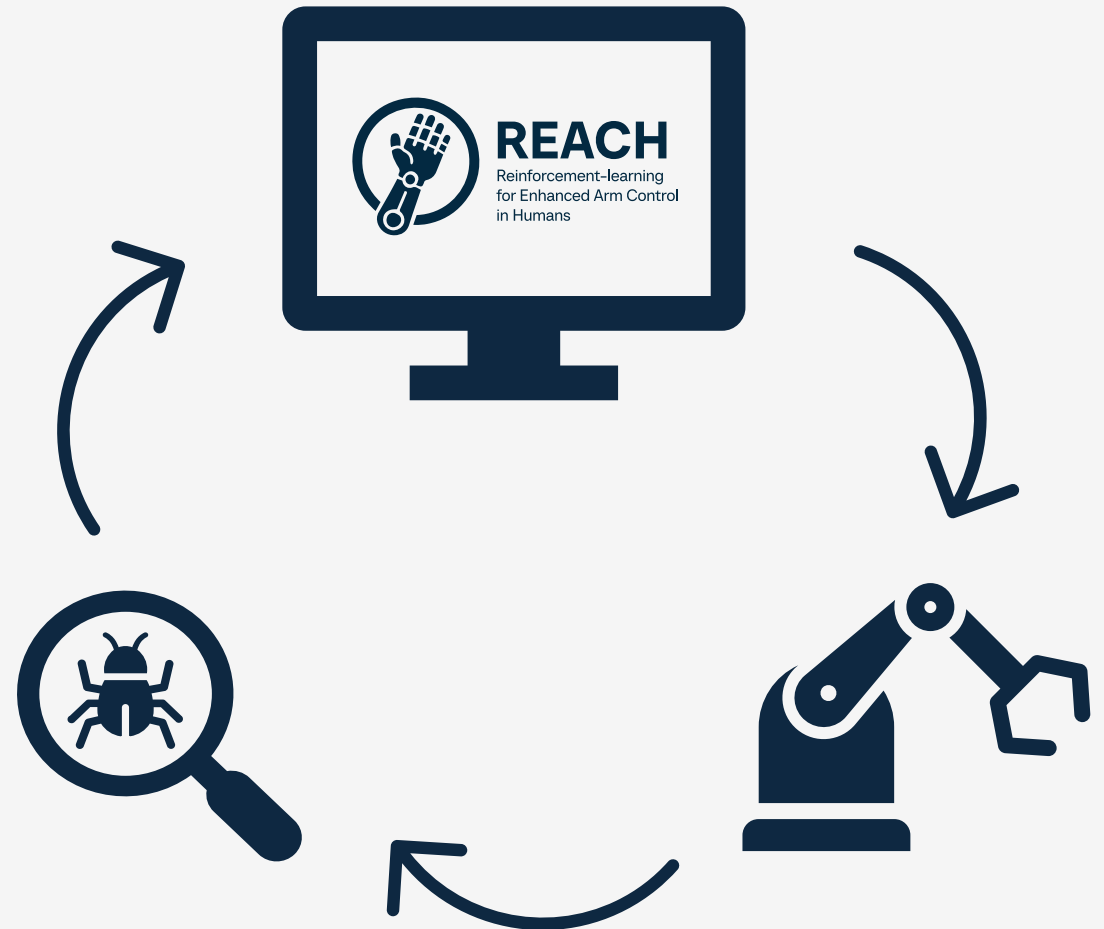
Solution Vision

Simulation based framework for developing and testing robotic devices

Integrated Reinforcement-Learning agent that learns adaptive control policies

Saved policies that can be replayed or transferred to real hardware

Computer vision capabilities for recognizing objects and hand gestures



Key Requirements

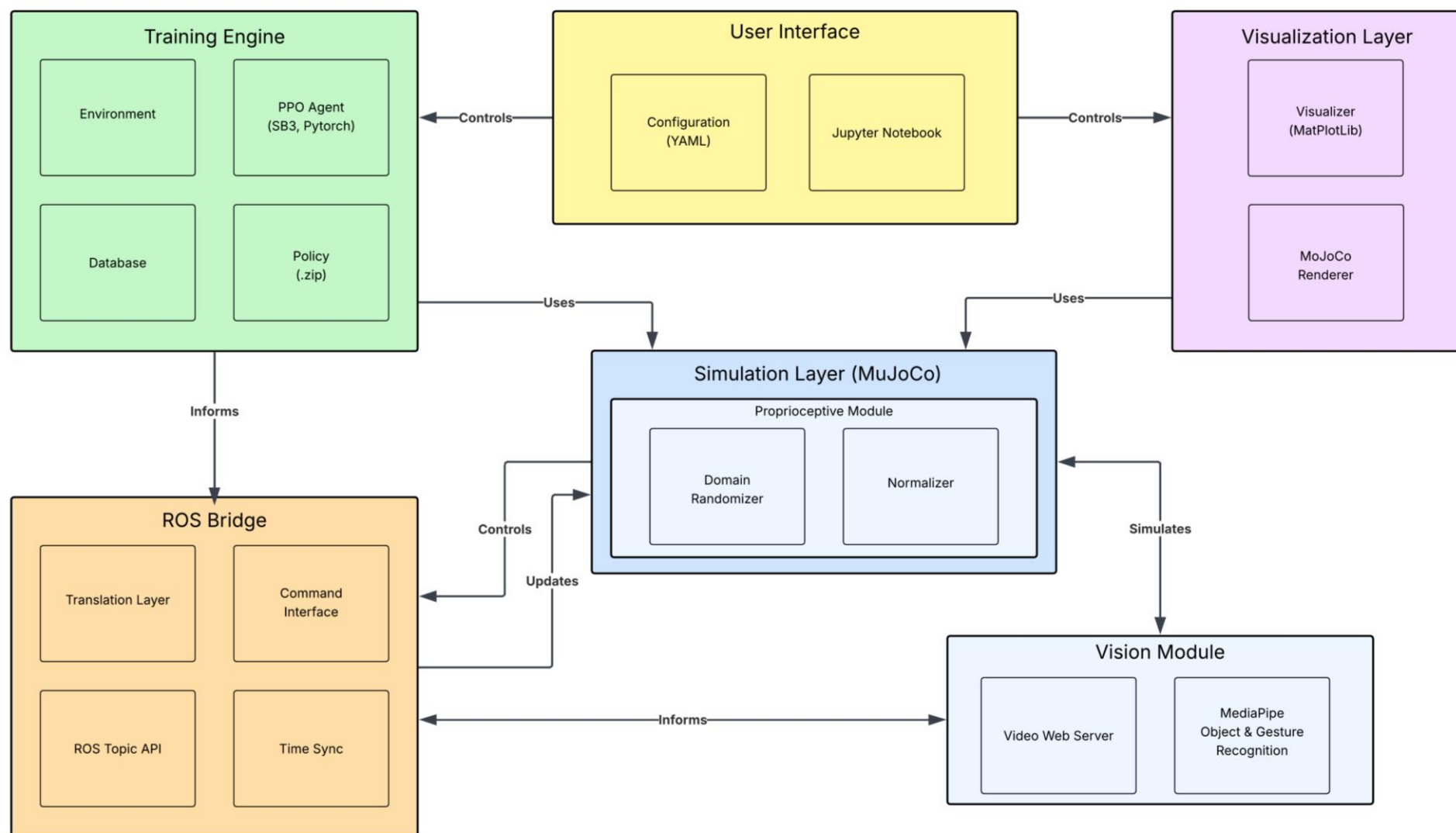
- Accurate simulation environment
- Easy to use YAML configuration
- Framework adaptability
- Reinforcement-Learning training loop
- Gesture recognition



- **Must Have**
- **Should Have**
- **Could Have**
- **Won't Have**

Software Architecture Overview

Tech Stack: Python • MuJoCo •
Gymnasium • Stable-Baselines3
(PPO) • PyTorch • YAML



Testing Plan

UNIT TESTS: CONFIGURATION LOADER, ENVIRONMENT SETUP, AND CORE UTILITIES

INTEGRATION TESTS: FULL PIPELINE (CONFIGURATION → ENVIRONMENT → MUJOCO → PPO → FINAL OUTPUTS)

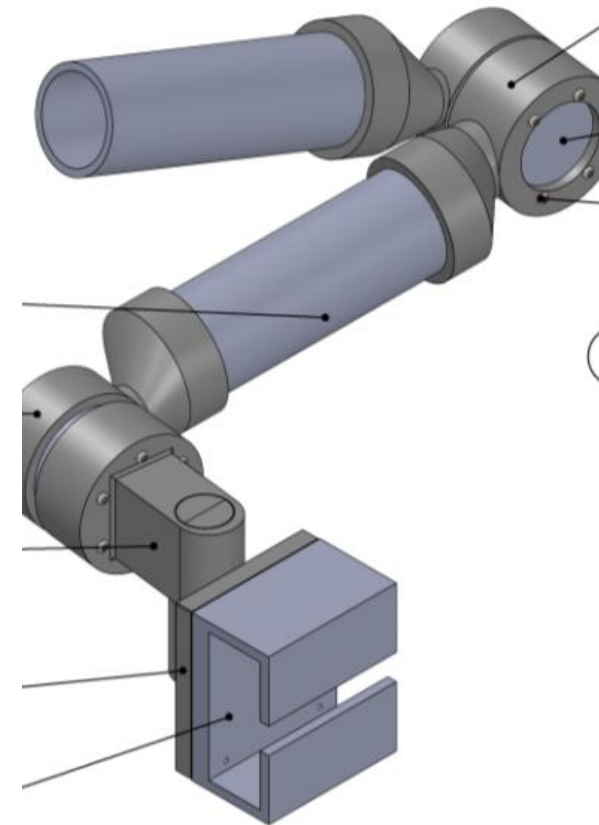
USABILITY TESTS: DEVELOPER WORKFLOWS AND USE-CASE READINESS

CONTINUOUS TESTING IN DEVELOPMENT CYCLE

LOGS, TEST RESULTS, AND CLIENT FEEDBACK GUIDE FIXES AND PRIORITIES

Alpha Demo 1 Features

- Working simulation environment
- Joints are active and stable
- Basic RL reaching task



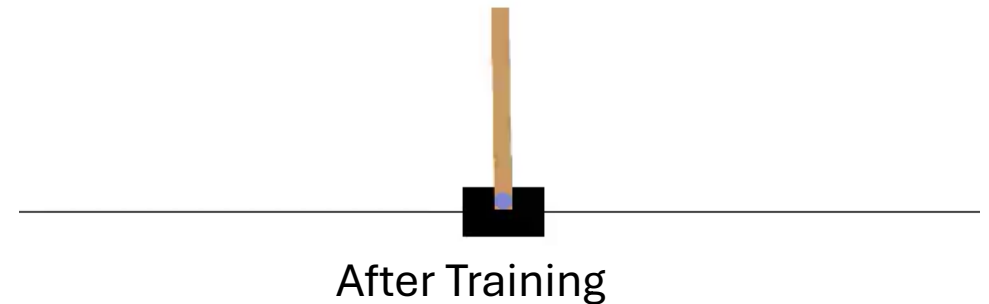
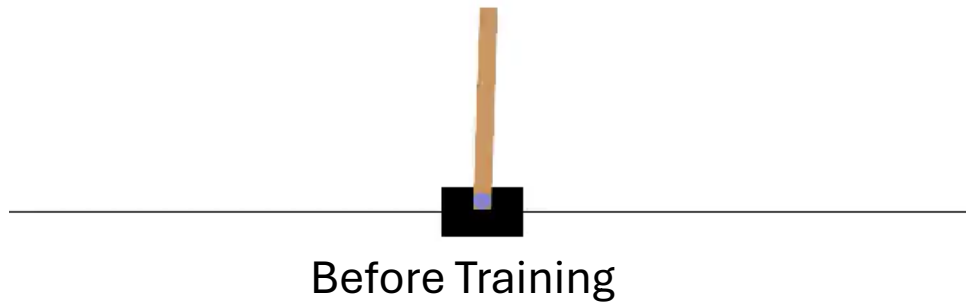
Simulation Environment

- We chose MuJoCo for accuracy and adaptability
- Widely used in robotics simulation
- Clean interface for reinforcement learning

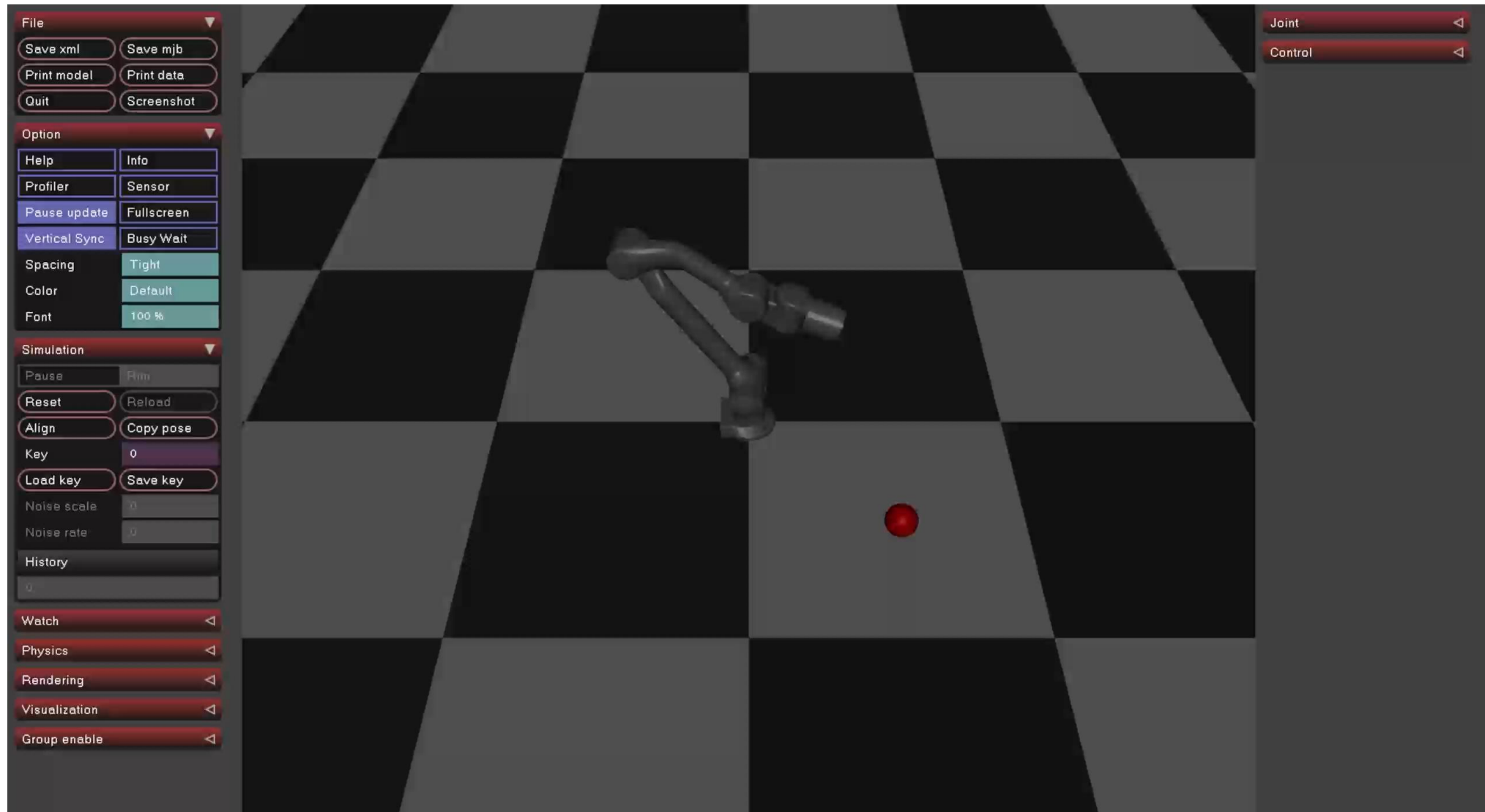


Reinforcement Learning

- StableBaselines3 offers great compatibility with MuJoCo
- PPO policies excel in learning multiple tasks

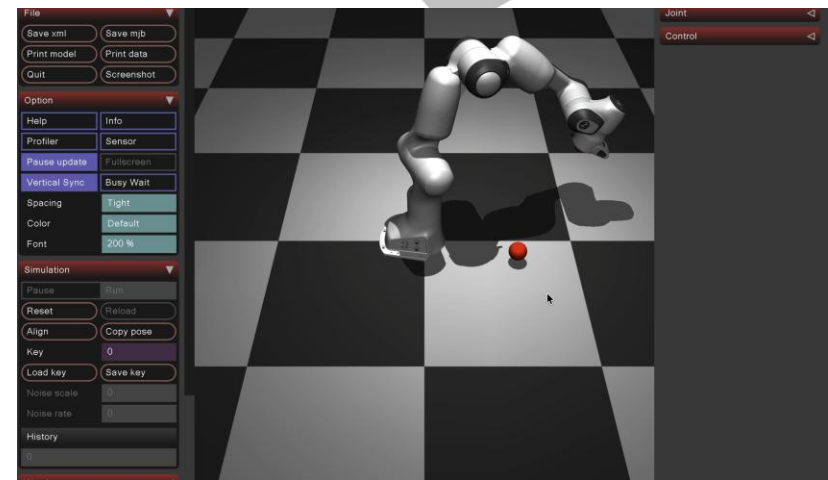
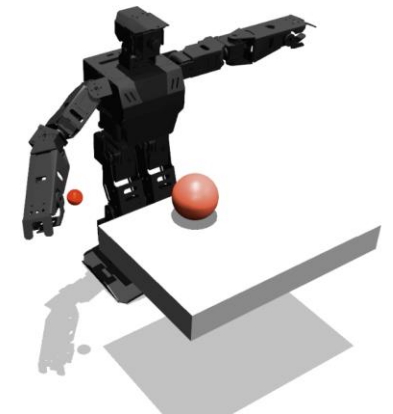


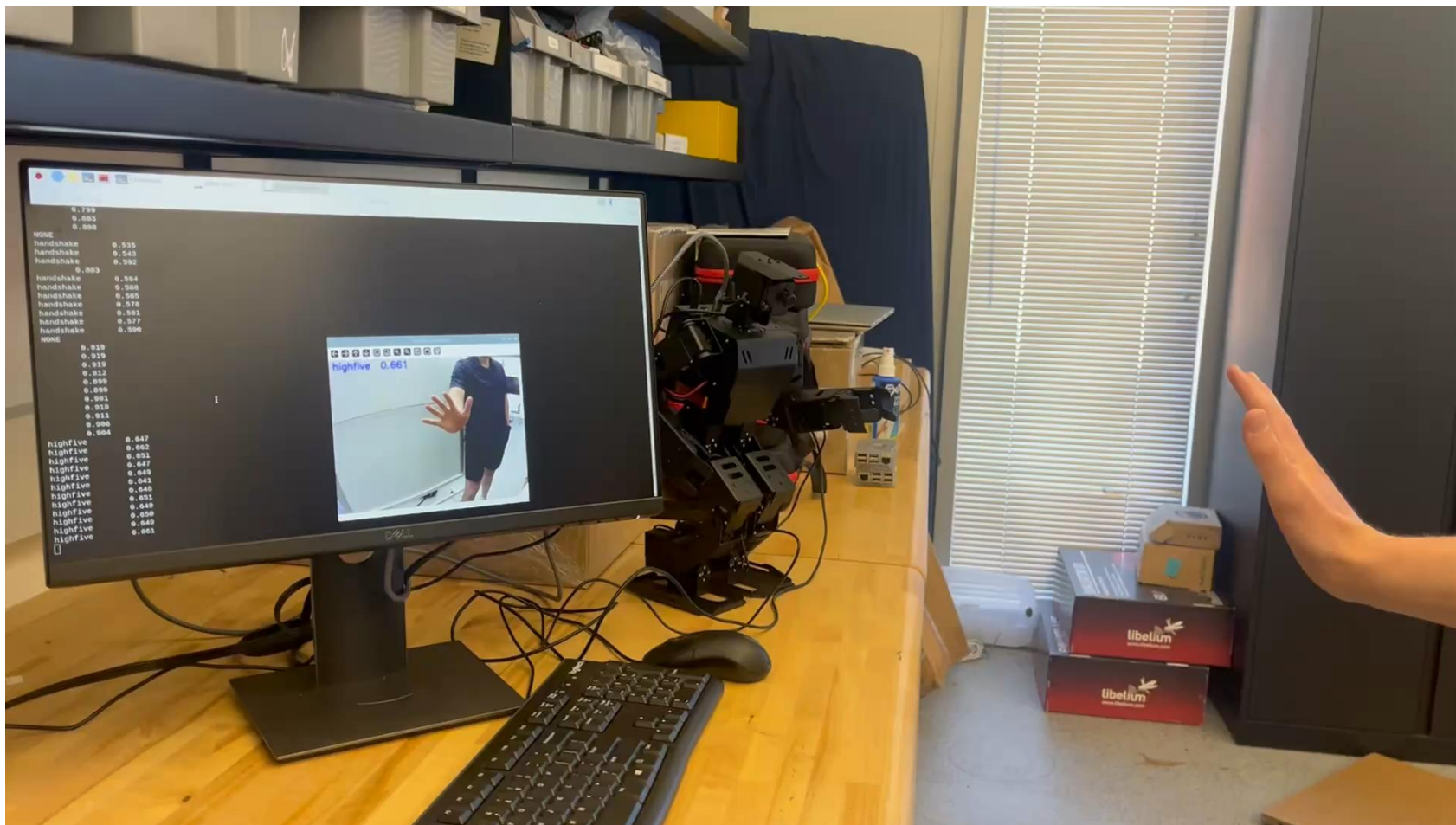
Reinforcement Learning



Alpha Demo 2 Features

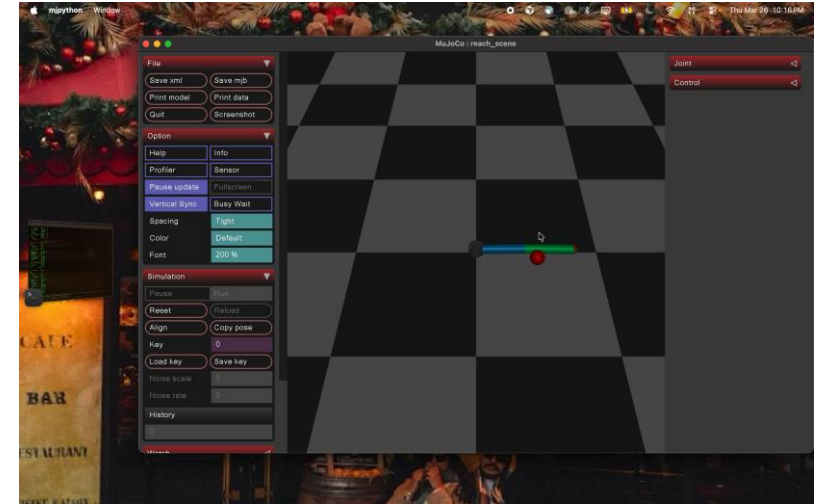
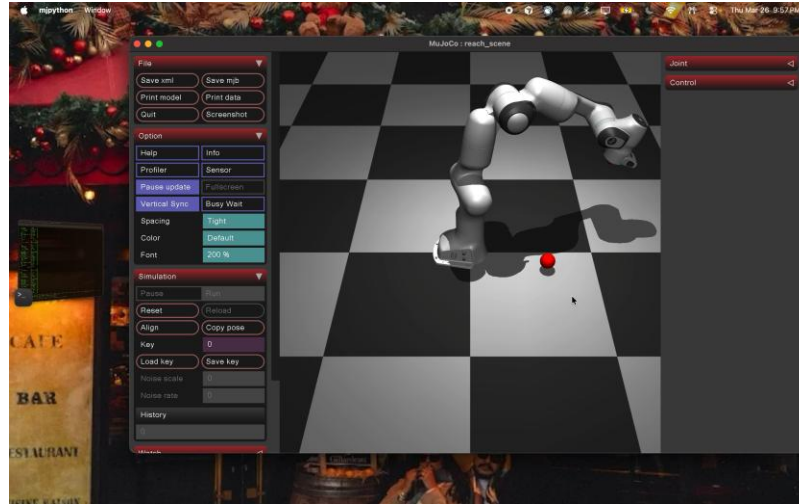
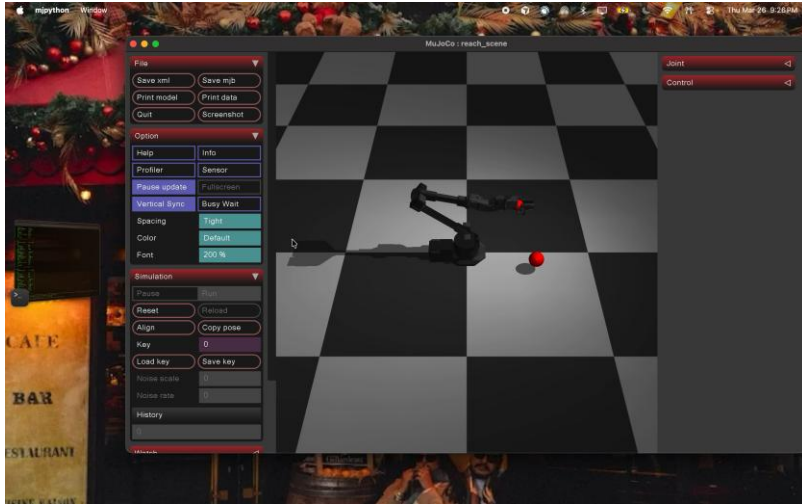
- Full body motion
- Framework adaptability
- Gesture recognition





Gesture Recognition

Multi-Robot, Multi-Scene Adaptability



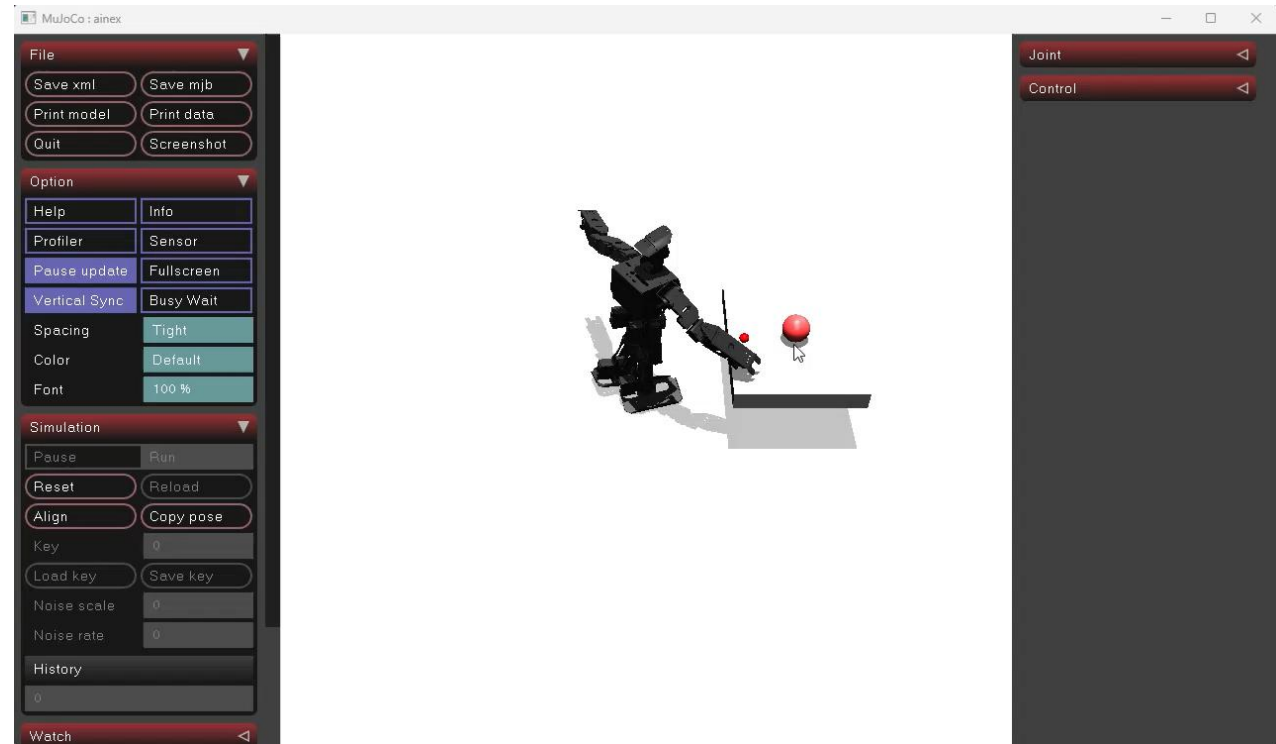
Drop-in robots - Standard folder layout and auto-discovery, train/run with one arm_id

YAML-driven - Rewards, steps, and run settings without editing core code

Same workflow everywhere - Add model then configure then train/run/evaluate

Ainex Reaching Task Improvement

- Involved all degrees of freedom in the reaching task
- Improved reward function
- Enhanced Ball Physics



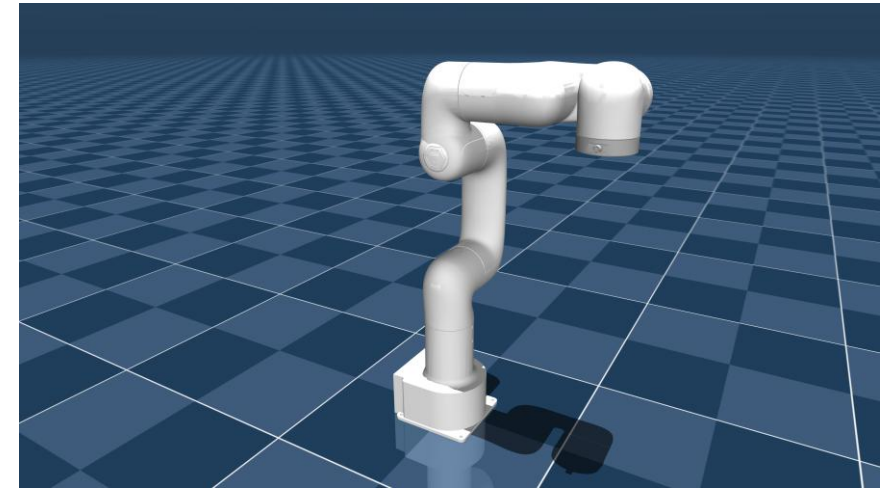
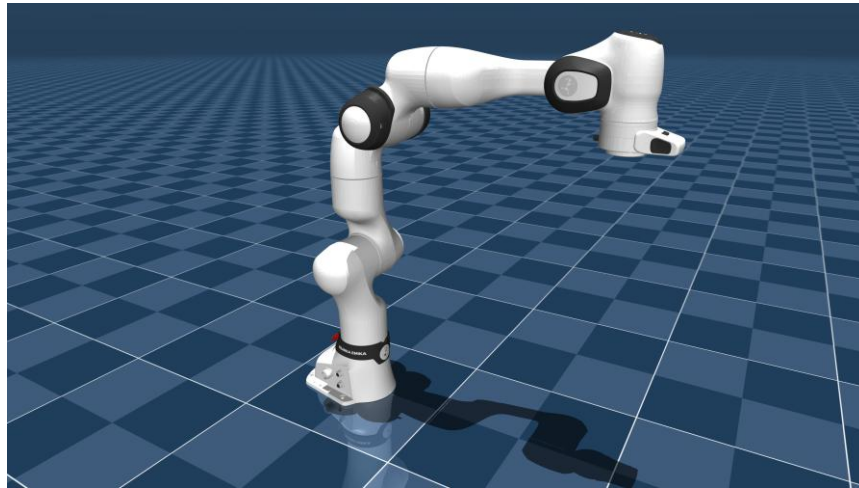
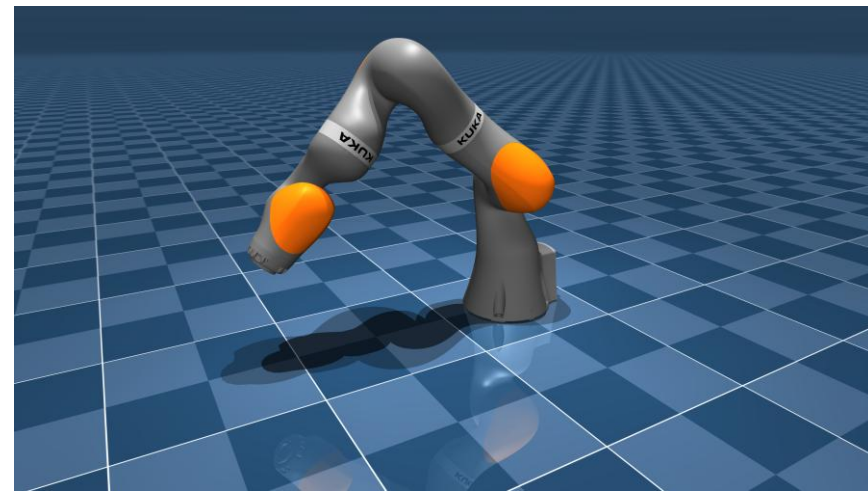
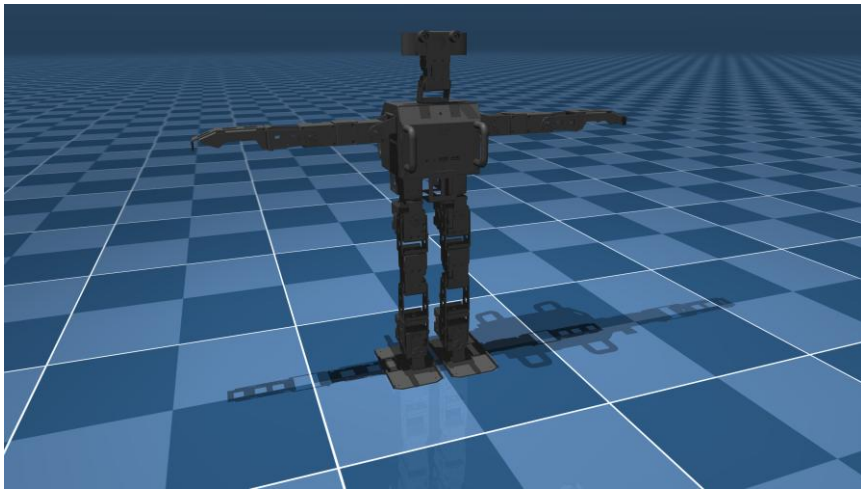
Acceptance Demo Features

- Response to gestures
- Enhanced adaptability
- Well defined documentation

```
78  # Project Structure
79
80  ""
81  reach/
82  |─ README.md
83  |─ requirements.txt
84  |─ pyproject.toml      # Packaging + CLI entry points
85  |
86  |─ assets/             # AINex Soccer assets (meshes, action groups)
87  |   |─ ainex/          # Robot models, URDF, MJCF, meshes
88  |   |─ action_groups/  # raw (.d6a) and csv exports
89  |   |─ config/
90  |   |   |─ arms.yaml    # Arm reach: arm_id, ball_mode, train/run settings
91  |   |   |─ render_run.yaml # Legacy: scene, model, policy for rendering
92  |   |   |─ ainex_render.yaml # AINex stand config
93  |   |   |─ ainex_reach.yaml # AINex reach config
94  |   |   |─ arms_loader.py  # Loads arms.yaml, resolves policy paths
95  |   |
96  |─ scripts/            # Default entry points
97  |   |─ train.py         # Train arm reach (delegates to arm_train_mac)
98  |   |─ run.py           # Run arm policy with viewer (delegates to run_simulation)
99  |─ policies/           # Trained arm policies (ppo_arms_<arm_id>_mac_<k>k.zip)
100 |
101 |─ cluster/
102 |   |─ train_monsoon.sh  # SLURM job for Monsoon HPC (auto-push on success)
103 |   |─ test_monsoon.sh   # Smoke test (local or Monsoon)
104 |─ tests/
105 |   |─ smoke_test.py     # Quick sanity check (imports, env, PPO)
106 |─ documentation/
107 |   |─ monsoon_setup.md  # Monsoon HPC setup and workflow
108 |   |─ demos/
109 |   |─ headshots/
110 |   |─ logos/
111 |   |─ system_design.md  # Architecture, assumptions, limitations
112 |
113 |─ envs/                 # Shared environments
114 |   |─ mujoco_arm_env.py
115 |
116 |─ tools/                # AINex Soccer conversion tools
117 |─ renders/              # Rendering and visualization scripts
118 |   |─ render_demo.py    # x86 / CUDA policy renderer
119 |   |─ render_demo_mac.py # macOS policy renderer (uses mpython)
120 |   |─ render_model.py   # x86 / CUDA model viewer
121 |   |─ render_model_mac.py # macOS model viewer (uses mpython)
122 |
123 |─ scenes/
124 |   |─ arms/             # Arm-only reach (YAML-driven, auto-discovery)
125 |   |   |─ env.py        # ArmReachEnv
126 |   |   |─ arm_registry.py # arm_id → config (ee_site, reach, etc.)
127 |   |   |─ arm_discovery.py # Auto-infer from MJCF
128 |   |   |─ scene_compose.py # Compose floor + ball + arm
129 |   |   |─ models/arms/   # MuJoCo Menagerie arms (panda, ur5e, aloha, etc.)
130 |   |   |   |─ training/  # arm_train_mac, run_simulation, eval_model
131 |   |─ ainex_soccer/     # AINex humanoid scene + training
132 |   |   |─ env.py
133 |   |   |─ models/
134 |   |   |─ policies/
135 |   |   |─ training/
```



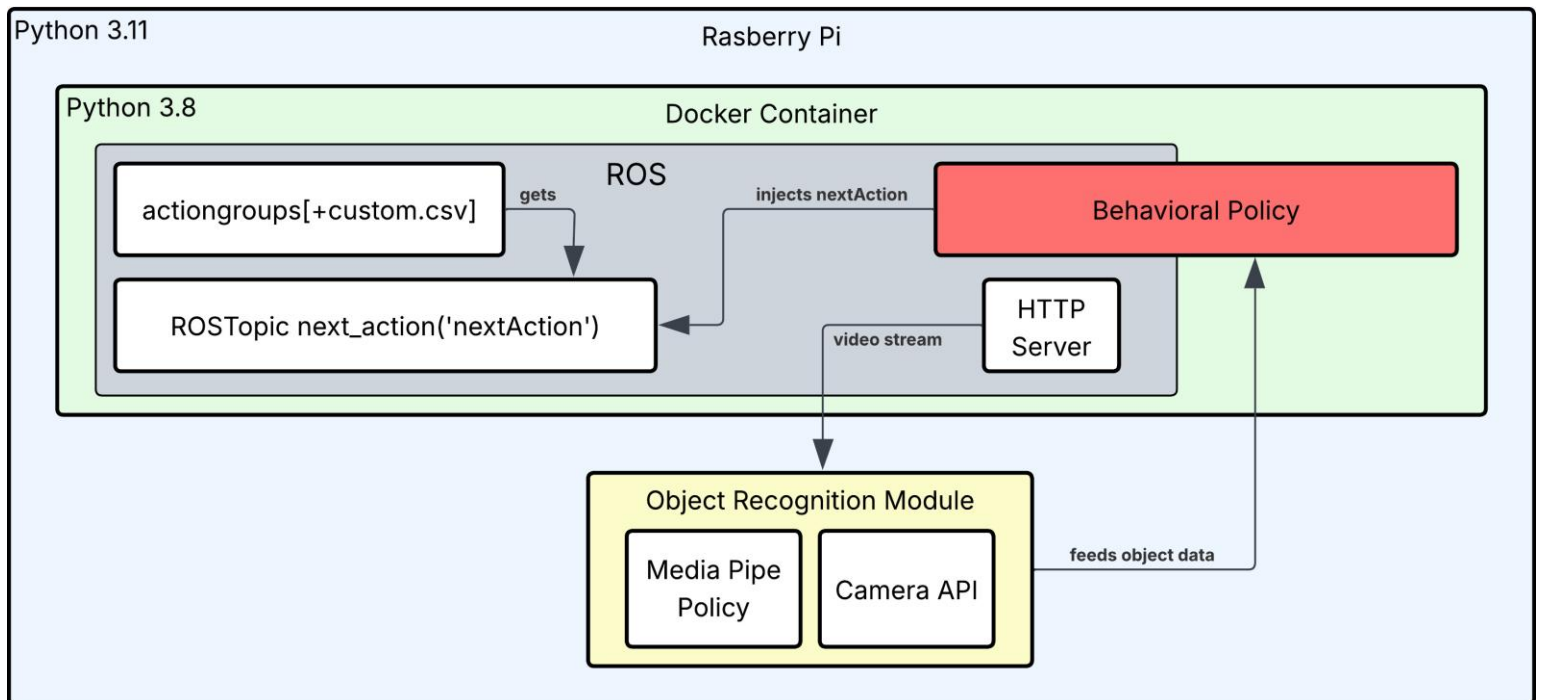

Gesture Recognition with Response



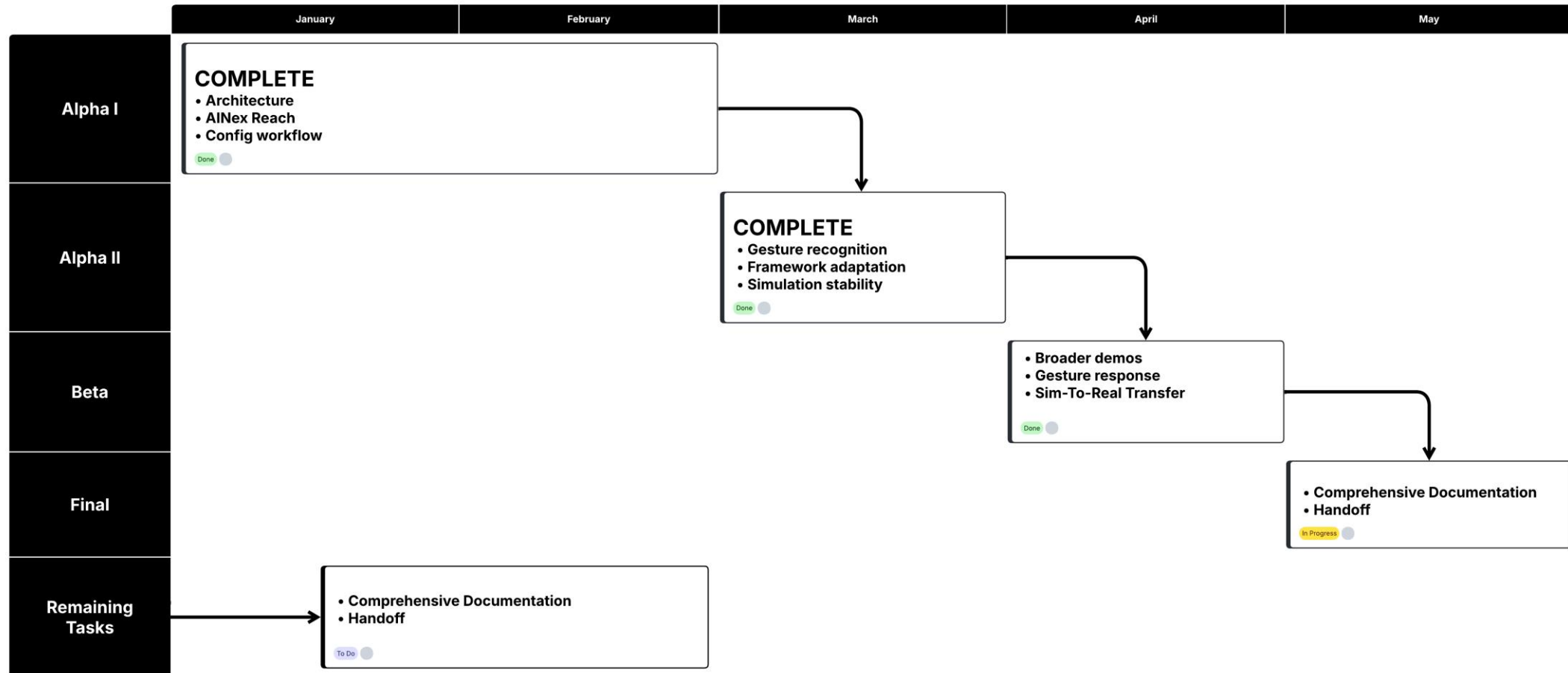
Extended Adaptability

Challenges & Resolutions

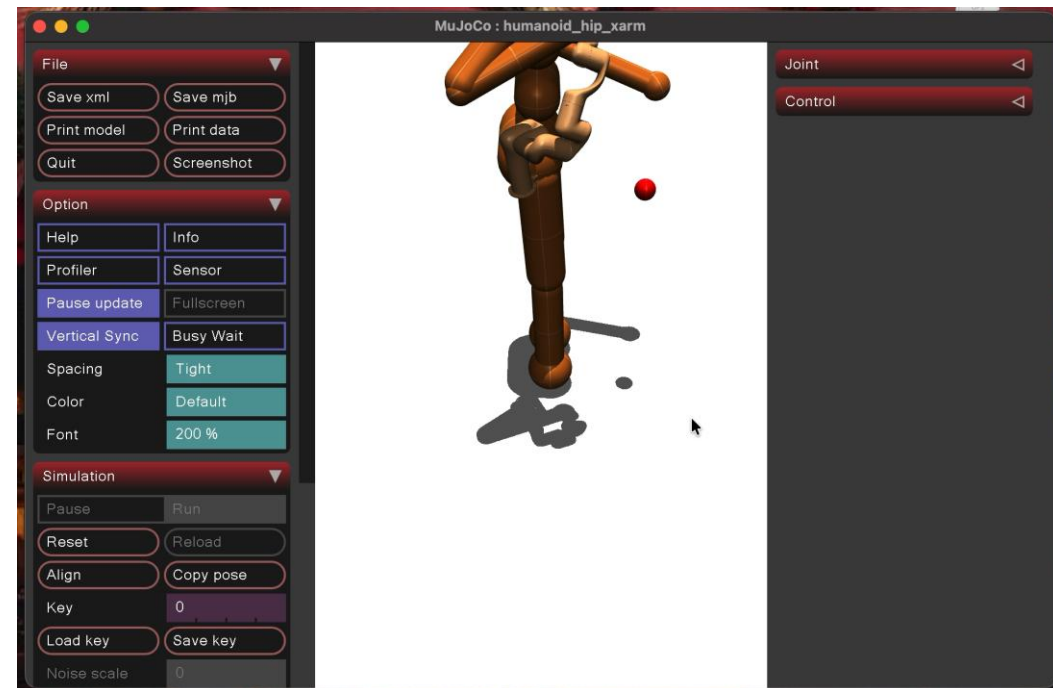
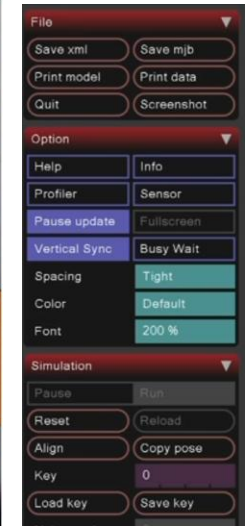
- Onboard Raspberry Pi had conflicting Python versions and Docker Constraints
 - Built local HTTP Server for access to video stream
- Handshake images for gesture recognition training are two-handed
 - We plan on creating our own image set for training
- Limited documentation on Ainex ROS implementation
- Project pivot due to lack of true rehabilitative robotic device



Progress



Future Work



Conclusion

Built: robust
reaching, adaptable
framework, gesture
recognition

Proved: reproducible
YAML workflow
across tasks

**Delivered
Value:** faster
research iteration for
biomechatronics

Next: Project
handoff and
documentation

Thank You

